

Como funciona o MCP

Status: documentação exploratória/descritiva para leitura técnica; não é contrato de implementação.

Data: 2026-06-15

Escopo: descrever como os MCPs Chatwoot customizados da Digi2B se apresentam para um runtime externo, com base em validação read-only nos MCPs de suporte Digi2B e Oral Unic Cuiabá.

Não escopo: definir arquitetura final do runtime, impor contratos de implementação, entregar credenciais, versionar dumps brutos, executar writes, alterar produção ou documentar dados reais de conversas/contatos.

Objetivo deste documento

Este documento explica a superfície MCP exposta pela versão customizada do Chatwoot usada pela Digi2B.

A intenção é ajudar uma equipe técnica a entender:

- como o MCP é configurado e identificado;
- quais famílias de tools existem;
- como os schemas de entrada são gerados;
- quais extensões customizadas aparecem além do Chatwoot padrão;
- quais tipos de dados podem aparecer em chamadas read-only;
- quais pontos exigem cuidado por envolverem escrita, dados sensíveis ou operações administrativas.

Este documento não define como o runtime final precisa ser implementado. Quando cita cuidados, o objetivo é explicar o comportamento observado do MCP e seus riscos naturais, não impor uma arquitetura específica.

Como a validação foi feita

A análise usou somente operações read-only ou de descrição:

- `hermes mcp test <server>` para validar conexão e contar tools expostas;
- `account_context` para confirmar account/server do MCP;
- `tool_describe` para schemas de tools representativas;
- `account_actions_list` com filtros `kanban`, `captain` e `uazapi` para entender a superfície REST customizada;
- leitura de documentação interna já existente sobre Chatwoot MCP e catálogo de MCPs por tenant.

Nenhuma tool de escrita/destrutiva foi chamada. Não foram coletados dumps de conversas, contatos, mensagens, tokens, URLs temporárias, webhooks ou anexos.

MCPs validados

MCP local no Hermes	Account Chatwoot	Nome seguro da account	Server remoto	Resultado
mcp-chatwoot-suporte	59	suporte Digi2B lAra	mcp-suporte	conectado; 132 tools descobertas
mcp-chatwoot-oral-unic-cuiaba	42	Oral Unic Cuiabá	oral-unic-cuiaba	conectado; 132 tools descobertas

Os dois MCPs usam transporte HTTP e endpoint no padrão:

None

https://app.digi2b.com/mcp/<account_id>/<slug>

A autenticação fica em header `Api-Access-Token`, configurado localmente por variável de ambiente. O valor do token não deve aparecer em documentação, logs ou exemplos.

Como o MCP aparece para o Hermes

A chave de configuração local não é igual ao prefixo real das tools.

Config local	Server/source MCP observado	Prefixo real das tools
mcp-chatwoot-suporte	mcp-mcp-chatwoot-supo rte	mcp_mcp_chatwoot_supo rte_*
mcp-chatwoot-oral-unic-cuiaba	mcp-mcp-chatwoot-oral -unic-cuiaba	mcp_mcp_chatwoot_oral _unic_cuiaba_*

Exemplos:

None

mcp_mcp_chatwoot_suporte_account_context
mcp_mcp_chatwoot_suporte_conversations_list
mcp_mcp_chatwoot_oral_unic_cuiaba_account_context

```
mcp_mcp_chatwoot_oral_unic_cuiaba_conversations_list
```

Observação: a nomenclatura tem normalização automática do Hermes/MCP. Por isso, para documentação e implementação, é melhor tratar o nome real da tool como metadado descoberto, e não como algo derivado manualmente do slug.

Tool de contexto da account

A tool mais simples é `account_context`.

Entrada:

```
JSON  
{}
```

Ela retorna metadados seguros do account e do MCP server quando o token está válido.
Exemplo sanitizado:

```
JSON  
{  
  "account": {  
    "id": 42,  
    "name": "Oral Unic - Cuiaba",  
    "locale": "pt_BR"  
  },  
  "mcp_server": {  
    "id": 3,  
    "name": "oral-unic-cuiaba",  
    "slug": "oral-unic-cuiaba",  
    "enabled": true  
  }  
}
```

Uso típico observado:

- confirmar qual account aquele MCP representa;
- validar se o endpoint/token correspondem ao tenant esperado;
- gerar evidência curta de conectividade.

Tools-base adicionadas pela camada MCP customizada

Além dos wrappers de endpoints Chatwoot, os MCPs analisados expõem algumas tools-base:

account_context

Retorna contexto da account e do MCP server.

account_actions_list

Lista actions REST disponíveis dentro do servidor Chatwoot/customizado.

Schema:

```
JSON
{
  "api_version": "v1|v2|global",
  "query": "string opcional"
}
```

Exemplos de filtros usados na análise:

```
JSON
{ "api_version": "v1", "query": "kanban" }
{ "api_version": "v1", "query": "captain" }
{ "api_version": "v1", "query": "uazapi" }
```

Comportamento observado:

- retorna path, controller, action e métodos HTTP disponíveis;
- revela a superfície customizada além das tools nomeadas;
- pode retornar muitos itens;
- não retorna credenciais por si só, mas descreve endpoints sensíveis que existem no backend.

account_action_call

Executa uma action REST por método/path.

Schema:

```
JSON
{
  "api_version": "v1|v2|global",
  "method": "GET|POST|PUT|PATCH|DELETE",
  "path": "/relative/path",
  "query": {},
  "body": {}
}
```

Comportamento observado:

- funciona como executor genérico de actions REST;
- a mesma interface pode representar GETs, POSTs, PATCH/PUT e DELETEs;
- amplia bastante a superfície disponível em relação às tools nomeadas.

Cuidados naturais:

- por ser genérica, essa tool não comunica intenção de negócio no próprio nome;
- quem consumir o MCP precisa distinguir leitura, escrita e deleção pelo método/path;
- em documentação compartilhável, é melhor registrar exemplos sintéticos em vez de paths com dados reais.

conversation_message_send

Helper customizado para envio de mensagem com texto e mídia.

Schema observado:

```
JSON
{
  "conversation_id": "<display_id>",
  "content": "texto opcional",
  "message_type": "outgoing|incoming",
  "private": true,
  "content_type": "text",
  "content_attributes": {},
  "media_urls": ["https://..."],
  "signed_ids": ["active_storage_signed_blob_id"]
}
```

Comportamento observado:

- permite mensagem privada ou pública, dependendo do campo `private`;
- aceita anexos por URLs externas ou signed blob ids;
- pode representar nota privada, resposta pública ou tentativa de mensagem inbound/outbound conforme payload e permissões do canal.

Cuidados naturais:

- `media_urls` e `signed_ids` podem conter URLs temporárias ou referências sensíveis;
- `private=false` pode gerar mensagem visível ao contato;
- WhatsApp/inboxes específicas podem rejeitar certos tipos de mensagem dependendo do canal e da API Chatwoot.

Padrão dos schemas gerados

A maioria das tools REST nomeadas segue um schema genérico.

Para listas/GETs sem path param:

```
JSON
{
  "query": {},
  "body": {}
}
```

Para endpoints com identificador:

```
JSON
{
  "id": "<id>",
  "query": {},
  "body": {}
}
```

Variações frequentes:

```
JSON
{ "conversation_id": "<conversation_id>", "query": {}, "body": {} }
{ "contact_id": "<contact_id>", "query": {}, "body": {} }
{ "team_id": "<team_id>", "query": {}, "body": {} }
{ "inbox_id": "<inbox_id>", "query": {}, "body": {} }
{ "portal_id": "<portal_id>", "query": {}, "body": {} }
```

Observação importante: mesmo em tools GET, o schema pode trazer **body** por causa da geração genérica. Isso não significa que o endpoint use body em GET; é apenas o formato uniforme do wrapper MCP.

Famílias de tools expostas

As famílias abaixo apareceram nos dois MCPs validados.

Account e perfil

- `account_get`
- `account_update`
- `profile_get`

Leitura simples de account/profile pode ser útil para diagnóstico. Updates já entram em operação administrativa.

Agents, teams e membros

- `agents_list/create/update/delete`
- `teams_list/get/create/update/delete`
- `team_members_list/add/update/delete`

Representam estrutura operacional do Chatwoot. Escritas alteram time, agentes e roteamento.

Inboxes e membros de inbox

- `inboxes_list/get/create/update`
- `inboxes_get_agent_bot`
- `inboxes_set_agent_bot`
- `inbox_members_list/create/update/delete`

Em ambiente customizado, leituras de inbox podem retornar configurações sensíveis de canal/provider. É uma família importante para entender canais conectados, mas precisa de sanitização antes de compartilhar outputs.

Agent bots

- `agent_bots_list/create/get/update/delete`

No tenant Oral Unic Cuiabá, a leitura read-only indicou agent bots associados a fluxos N8n, incluindo bot principal e bot de campanha. Endpoints e segredos foram redigidos na análise.

Contatos

- `contacts_list`
- `contacts_create`
- `contacts_get`
- `contacts_update`
- `contacts_delete`
- `contacts_search`
- `contacts_filter`
- `contacts_conversations`
- `contacts_create_contact_inbox`
- `contacts_contactable_inboxes`
- `contacts_merge`
- `contact_labels_list`
- `contact_labels_set`

`contacts_filter` usa POST, mas semanticamente funciona como busca/filtro. Contatos podem conter PII; por isso, não foram listados no documento.

Conversas

- `conversations_meta`
- `conversations_list`
- `conversations_create`
- `conversations_filter`
- `conversations_get`
- `conversations_update`
- `conversations_toggle_status`
- `conversations_toggle_priority`
- `conversations_set_custom_attributes`
- `conversations_get_labels`
- `conversations_set_labels`
- `conversations_reporting_events`
- `conversation_assignments_assign`

`conversations_filter` também usa POST para filtro. Conversas podem conter histórico, status, labels e dados pessoais.

Mensagens

- `messages_list`
- `messages_create`
- `messages_delete`
- `conversation_message_send`

Mensagens são a família com maior sensibilidade de conteúdo. `messages_create` e `conversation_message_send` podem gerar side effects visíveis ou privados, dependendo do payload.

Labels

- `conversations_get_labels`
- `conversations_set_labels`
- `contact_labels_list`
- `contact_labels_set`

Observação sobre comportamento: endpoints de set labels em Chatwoot frequentemente trabalham com a lista de labels enviada. Se a intenção for apenas adicionar/remover uma label, a integração precisa entender se o endpoint substitui a lista completa ou faz merge.

Custom attributes e filtros

- `custom_attributes_list/create/get/update/delete`
- `custom_filters_list/create/get/update/delete`
- `conversations_set_custom_attributes`

Custom attributes podem ser usados para estado operacional, mas também podem conter dados sensíveis ou acoplar automações.

Webhooks e integrações

- `webhooks_list/create/update/delete`
- `integrations_list_apps`
- `integrations_create_hook`
- `integrations_update_hook`
- `integrations_delete_hook`

Leituras ajudam a entender integrações configuradas. Escritas alteram comportamento do Chatwoot e integrações externas.

Automation rules e canned responses

- `automation_rules_list/create/get/update/delete`
- `canned_responses_list/create/update/delete`

No recorte analisado do tenant Oral Unic Cuiabá, automation rules nativas vieram vazias, indicando que a automação principal parece estar fora delas, via agent bots/N8n e Kanban/templates.

Reports e audit logs

- `reports_account_overview`
- `reports_account_summary`
- `reports_agent_summary`
- `reports_conversation_metrics`
- `reports_v2_overview`
- `reports_v2_agents`
- `reports_v2_inboxes`
- `reports_v2_teams`
- `reports_v2_labels`
- `audit_logs_list`

Reports são leitura, mas podem revelar volume operacional, filas, labels e desempenho. Audit logs podem revelar atores, eventos e metadados sensíveis.

Help center

- `help_center_portals_list/create/update`
- `help_center_categories_list/create`
- `help_center_articles_list/search/get/create`

Não foi foco da análise, mas aparece como parte da superfície MCP.

Extensões customizadas observadas

Além das famílias esperadas do Chatwoot, a versão customizada expõe superfícies adicionais.

Kanban/funnels

Tools nomeadas:

- `kanban_boards_list/create/get/update/delete`
- `kanban_steps_list/create/get/update/delete`
- `kanban_tasks_list/create/get/update/delete`
- `kanban_tasks_move`
- `kanban_preferences_get`

Actions REST vistas por `account_actions_list`:

- `/kanban/funnels`
- `/kanban/stages`
- `/kanban/items`
- `/kanban_items`
- `/kanban_items/:id/move`
- `/kanban_items/:id/move_to_stage`
- `/kanban_items/bulk_move_items`
- `/kanban_items/bulk_assign_agent`
- `/kanban_items/bulk_set_priority`
- `/kanban_items/import`
- `/kanban_items/export`
- endpoints de checklist, notes, attachments e reports.

No tenant Oral Unic Cuiabá, a leitura resumida encontrou boards/funnels customizados como `ATENDIMENTO`, `RELACIONAMENTO`, `Follow Avaliação` e `Follow de Lead`, com stages/templates. Não foram copiados dumps de templates para este documento.

UAZAPI / WhatsApp custom

Actions REST vistas por discovery:

- `/inboxes/:id/connect_uazapi_instance`
- `/inboxes/:id/disconnect_uazapi_instance`
- `/inboxes/:id/recreate_uazapi_instance`
- `/inboxes/:id/uazapi_instance_status`
- `/inboxes/:inbox_id/whatsapp_uazapi/connect`
- `/inboxes/:inbox_id/whatsapp_uazapi/disconnect`
- `/inboxes/:inbox_id/whatsapp_uazapi/delete`
- `/inboxes/:inbox_id/whatsapp_uazapi/recreate`
- `/inboxes/:inbox_id/whatsapp_uazapi/subscribe_webhooks`
- `/inboxes/:inbox_id/whatsapp_uazapi/update_chats_settings`
- `/inboxes/:inbox_id/whatsapp_uazapi/get_chats_settings`
- `/inboxes/:inbox_id/whatsapp_uazapi/instance_status`

Essa superfície parece controlar conexão/configuração de instâncias WhatsApp UAZAPI. É útil para entender a customização, mas não é equivalente a uma simples leitura de conversa.

Captain / AI

Tools e actions relacionadas:

- `captain_tasks_rewrite`
- `captain_tasks_summarize`
- `captain_tasks_reply_suggestion`
- `captain_tasks_follow_up`
- `captain_tasks_label_suggestion`
- assistants, assistant responses, scenarios, documents, custom tools, copilot threads/messages, preferences e MCP servers.

Actions vistas por discovery incluem criação/edição/deleção de assistants, documents, custom tools e MCP servers do Captain, além de connect/disconnect/refresh.

Ponto observado: no inventário do subagente, uma tool de Captain MCP servers apareceu em catálogo/schema, mas uma chamada específica falhou como `Tool not found`. Isso indica que a superfície descoberta e a tool materializada podem divergir em alguns pontos; a documentação de consumo deve prever essa possibilidade.

Observações específicas do tenant Oral Unic Cuiabá

A análise read-only apontou, de forma sanitizada:

- account `42`, nome `Oral Unic - Cuiaba`, locale `pt_BR`;
- MCP server remoto `oral-unic-cuiaba`, enabled `true`;
- 132 tools descobertas pelo Hermes;
- duas inboxes WhatsApp observadas no resumo do subagente:

- uma via WhatsApp Cloud;
- uma via WAHA/campanha;
- agent bots relacionados a fluxos N8n do tenant, incluindo bot principal e bot de campanha Plantão de Junho;
- teams vazios no recorte lido;
- custom attributes vazios no recorte lido;
- automation rules nativas vazias no recorte lido;
- webhooks account-level vazios no recorte lido;
- help center sem portais no recorte lido;
- Kanban/funnels customizados com boards de atendimento/relacionamento/follow.

Esses dados servem para entender a estrutura do MCP e da account, não como documentação operacional completa do tenant.

Observações específicas do MCP de suporte

A análise read-only apontou, de forma sanitizada:

- account **59**, nome seguro **suporte Digi2B IAra**, locale **pt_BR**;
- MCP server remoto **mcp-suporte**, enabled **true**;
- 132 tools descobertas pelo Hermes;
- superfície semelhante à do tenant Oral Unic Cuiabá;
- presença das mesmas famílias principais: conversas, mensagens, contatos, inboxes, agents/teams, webhooks, reports, Captain, Kanban e actions customizadas.

O MCP de suporte é útil como referência técnica porque concentra a mesma superfície customizada sem ser um tenant clínico específico.

Como interpretar leitura vs escrita

O MCP não separa toda a superfície em nomes semanticamente seguros. A diferença entre leitura, escrita e deleção aparece em três lugares:

1. nome da tool (***_list**, ***_get**, ***_create**, ***_update**, ***_delete**, **toggle_***, **set_***);
2. método HTTP quando a ação vem de **account_actions_list/account_action_call**;
3. payload enviado em **body**.

Exemplos:

Exemplo	Natureza provável
account_context	leitura de contexto
conversations_list	leitura sensível

Exemplo	Natureza provável
<code>messages_list</code>	leitura sensível de conteúdo
<code>contacts_filter</code>	leitura/filtro, apesar de usar POST
<code>messages_create</code>	escrita de mensagem
<code>conversation_message_send</code> com <code>private=true</code>	nota privada ou mensagem privada, conforme endpoint/canal
<code>conversation_message_send</code> com <code>private=false</code>	mensagem pública/outbound possível
<code>conversations_set_labels</code>	escrita de labels
<code>conversations_toggle_status</code>	alteração de estado operacional
<code>kanban_tasks_move</code>	alteração de Kanban
<code>webhooks_delete</code>	operação destrutiva/admin
<code>account_action_call</code> com DELETE	operação genérica destrutiva

Tipos de dados que podem aparecer nas respostas

Mesmo chamadas read-only podem retornar dados sensíveis. Exemplos por família:

Família	Dados possíveis
<code>contacts_*</code>	nomes, telefones, e-mails, identificadores, atributos customizados
<code>conversations_*</code>	status, labels, assignee, inbox, timestamps, custom attributes
<code>messages_*</code>	conteúdo de conversa, anexos, sender, private/public, timestamps
<code>inboxes_*</code>	provider config, canal, phone metadata, agent bot, configurações de working hours
<code>agent_bots_*</code>	URLs de webhook, segredos/configs dependendo da serialização
<code>webhooks_*</code>	URLs e subscriptions
<code>reports_*</code>	métricas operacionais e volumes

Família	Dados possíveis
<code>audit_logs_list</code>	eventos administrativos e atores
<code>kanban_*</code>	boards, stages, templates, notas, checklist, tarefas
<code>captain_*</code>	conteúdo processado por IA, documents, assistants, custom tools

Por isso, os exemplos deste documento são resumidos e sanitizados.

Diferenças em relação ao Chatwoot padrão básico

A superfície observada não é apenas a API pública básica do Chatwoot. A versão customizada expõe:

- wrapper MCP por account/tenant;
- helper genérico de discovery (`account_actions_list`);
- executor genérico de action REST (`account_action_call`);
- Kanban/funnels customizados;
- integração UZAPI/WhatsApp custom;
- Captain/AI com documents, assistants, custom tools e MCP servers;
- helper de mensagem com mídia (`conversation_message_send`);
- reports e audit logs via tools;
- endpoints de agent bots/N8n visíveis em leituras estruturais.

Essa diferença é relevante para qualquer equipe que consuma o MCP, porque a superfície é mais ampla do que “listar conversas e enviar mensagens”.

Cuidados de documentação e compartilhamento

Ao documentar ou entregar evidências sobre esse MCP, evitar:

- tokens e headers;
- URLs reais de webhooks;
- payload bruto de conversations/messages/contacts;
- dumps completos de `account_actions_list`;
- dumps de inboxes/agent bots sem sanitização;
- signed ids, URLs temporárias, blob keys ou anexos;
- dados reais de pacientes/leads;
- templates completos se contiverem conteúdo comercial sensível.

Formatos seguros para documentação:

- contagem de tools;
- lista de famílias;

- schemas genéricos;
- paths sem IDs reais;
- exemplos sintéticos;
- hashes/contagens quando for preciso registrar drift;
- nomes de tenants/accounts apenas quando autorizado para documentação interna.